

Implementation-Only Research Review of `experiments_related`

Evidence-backed review of the uploaded implementation export

Generated July 7, 2026

Abstract

This document reviews the uploaded implementation-only export of `experiments_related`. The review is limited to code, shell launchers, configuration files, and tests present in the archive. The archive intentionally excludes large outputs, checkpoints, logs, prediction tensors, and generated artifacts, so this review does not validate any numerical result or claim that a run completed. Code claims cite exact paths and, where inspected, line ranges. External method names are grounded with bibliographic sources for V-JEPA, V-JEPA 2, DINOv3, DPT, OccuFly, CGFormer, FoundationSSC, and VoxFormer [1, 2, 4, 6, 7, 8, 9, 11].

1 Scope, evidence policy, and archive inventory

Scope. The evidence base is the extracted archive rooted at `experiments_related`. All statements about implemented behavior are based on files in that export. If a connection is inferred from naming, wrapper code, or a shared filesystem convention rather than a direct import or explicit call, it is marked as an inference.

Inventory. The export contains 599 implementation-oriented files: 465 Python files, 49 shell scripts, 54 YAML files, 25 JSON files, one TOML file, and three text files. The top-level directories are `scripts`, `vjepa21video_scripts`, `scripts_ssc`, `scripts_segmentation`, `tests`, and `forpaper_report`. A hash-based duplicate scan of the extracted archive found many exact duplicate groups; the `forpaper_report` directory is therefore treated as a snapshot/mirror rather than a separate independent implementation family.

Important negative evidence. The archive does not include trained checkpoint files, raw run logs, TensorBoard logs, prediction tensors, generated depth maps, or result tables other than code that writes those files. Therefore this review can identify what the implementation is designed to support, but not whether any reported metric was achieved.

2 Project structure map

Family	Primary paths	Implemented role
Monocular depth estimation (MDE)	<code>experiments_related/scripts/03_train_vjepa_linear_depth.py</code> ; <code>06_vjepa21_dinov3_linearprobe_occufly.py</code> ; <code>07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py</code> ; <code>scripts/dpt_decoder/*.py</code>	Frozen V-JEPA/V-JEPA2.1 depth probes; DINOv3-style depth heads; DPT multilayer decoders; FiLM conditioning by altitude and camera metadata; SFP/final-layer variants; constant-depth and diagnostic baselines. The linear probe script declares a supervised linear depth probe on frozen V-JEPA2.1 features and an OccuFly split/layout <code>experiments_related/scripts/03_train_vjepa_linear_depth.py</code> (lines 3–16, 38–45).
Video depth	<code>experiments_related/vjepa21video_scripts/src/vjepa21video_depth</code> ; <code>vjepa21video_dptfilm_attention/src</code>	Spatio-temporal linear probing (ST-LP) and video DPT-FiLM with dense temporal attention (DTA). Clip construction modes are <code>repeated_stlp</code> , <code>real_stlp</code> , and <code>shuffled_stlp</code> in the ST-LP constants <code>experiments_related/vjepa21video_scripts/src/vjepa21video_depth/constants.py</code> (lines 24–36).
V-JEPA2.1 RGB/video adaptation	<code>experiments_related/scripts/08_vjepa21_finetune_occufly.py</code> ; <code>11_vjepa21_finetune_occufly_video.py</code> ; wrappers <code>10_*</code> and <code>11_vjepa21_finetuned_*</code>	Self-supervised OccuFly-domain V-JEPA2.1 adaptation on RGB images or materialized videos. The RGB finetuning script states that it uses RGB frames only and does not use depth maps or labels <code>experiments_related/scripts/08_vjepa21_finetune_occufly.py</code> (lines 2–9, 302–331). The video finetuning script materializes MP4 sequences from image folders and also states that labels/depth maps are unused <code>experiments_related/scripts/11_vjepa21_finetune_occufly_video.py</code> (lines 2–13, 176–240, 380–391).

Family	Primary paths	Implemented role
Semantic segmentation	<code>experiments_related/scripts_segmentation/dataset_download/download_voc12.sh;</code> <code>scripts_segmentation/linear_pipeline/*.sh;</code> <code>occufly_21class_label_map.json</code>	Thin launcher layer around DINOv3 tools for VOC12 and OccuFly linear sweeps, plus a 21-name OccuFly label map. The VOC script downloads and verifies VOC2012; the two sweep shell scripts exec <code>dinov3/tools/run_vjepa21_*_linear_sweep.py</code> .
Semantic scene completion: simple/scaffold	<code>experiments_related/scripts_ssc/vjepafoundationssc</code>	Early V-JEPA-to-SSC pipeline with multi-layer V-JEPA backbone, context head, DPT-FiLM depth-probability head, frustum voxel pooling, and a simple 3D decoder. This family appears as a local scaffold rather than the most advanced FoundationSSC integration.
Semantic scene completion: FoundationSSC hybrid	<code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc</code>	Most complete SSC implementation in the export: V-JEPA RGB/video context, DPT-FiLM or pose-aware MVS geometry branch, FoundationSSC hybrid view transformation, FoundationSSC decoder, OccuFly SSC dataset loader, losses, metrics, training, evaluation, and visualization. The model constructor explicitly accepts <code>input_modein{rgb,video}</code> and <code>geometry_branchin{dpt_depth,pose_mvs}</code> <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py</code> (lines 232–264).

Family	Primary paths	Implemented role
CGFormer / Aero-JEPA SSC ablations	<code>experiments_related/scripts_</code> <code>ssc/cgformer/aero_jepa_ssc_</code> <code>project</code>	CGFormer-compatible ablation project with configs for baseline, M1–M6 variants, altitude lifting, depth teacher losses, V-JEPA feature caching/distillation, latent view consistency, masked-3D losses, output consistency, and Table 3 generation. The config module enumerates altitude and V-JEPA teacher variants <code>experiments_related/scripts_</code> <code>ssc/cgformer/aero_jepa_ssc_</code> <code>project/src/aero_jepa_ssc/</code> <code>config.py</code> (lines 13–69).
Diagnostics, visualization, and reporting	<code>experiments_related/scripts/</code> <code>diagnostics;</code> <code>scripts/visualization;</code> <code>scripts_ssc/vizualization;</code> <code>scripts_ssc/*/visualize_*.py;</code> <code>paper_mde_grouped_tables.py</code>	Constant-depth baselines, scale/prediction-shift checks, depth viewers, SSC viewers, representation visualizers, and scripts for result-table aggregation.
Tests	<code>experiments_related/tests/*.py;</code> <code>scripts_ssc/cgformer/aero_</code> <code>jepa_ssc_project/tests/*.py</code>	Unit tests for mask-aware depth resizing, SSC metric semantics, and CGFormer/Aero-JEPA components such as depth teachers, masked 3D, latent view, output consistency, Table 3 provenance, and V-JEPA feature caching.

3 Data flow reconstruction

3.1 OccuFly data conventions

The MDE scripts expect OccuFly scene/altitude folders of the form:

```
scene_XX/{30,40,50}/images/visual/*.png
scene_XX/{30,40,50}/depth_maps/*
scene_XX/calibration.txt
```

The frozen linear-probe script matches RGB and depth files by stem under `images/visual` and `depth_maps`; it raises an error when no RGB/depth pairs are found `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 104–147). The same script defines the default train/validation/test split as scenes 01–05, 06–07, and 08–09 respectively `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 9–12). The DINOv3-style linear probe uses the same fixed split and layout `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py` (lines 60–63, 196–220).

Depth maps are handled as either NumPy arrays or image-like encodings in the inspected scripts. Mask-aware resizing is implemented by interpolating a valid-pixel weighted depth numerator and a valid mask denominator, rather than interpolating invalid depth values directly `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 74–88); the same pattern appears in the DINOv3-style linear probe `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_`

occufly.py (lines 128–142) and is tested in `experiments_related/tests/test_mask_aware_depth_resize.py`.

Camera metadata used by FiLM depth models is a five-vector:

$$m = [h_{norm}, f_x/W, f_y/H, c_x/W - 0.5, c_y/H - 0.5].$$

The vector is constructed from altitude and `calibration.txt` intrinsics in the DPT-FiLM MDE implementation `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py` (lines 420–450) and in the video-depth data layer `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/occufly_data.py` (lines 117–146).

3.2 MDE flow

The basic MDE data path is:

OccuFly RGB + depth

- > OccuFly depth dataset with valid mask and metadata
- > frozen V-JEPA2.1 image features
- > depth head / DPT decoder / FiLM DPT decoder
- > masked depth loss
- > depth metrics and checkpoint selection by validation RMSE

The linear probe loads a V-JEPA2.1 encoder from a local `vjepa2` repo or from `facebookresearch/vjepa2`, freezes it, and trains only a `Conv2d(D,1,1)` dense linear head `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 260–284, 320–352, 474–493). Static image inputs are repeated along a temporal dimension if the encoder expects video input `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 408–417). The optimizer is AdamW over head parameters only, and best validation RMSE plus final test RMSE are written to metrics outputs `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 636–670).

The DINOv3-style linear probe imports DINOv3 depth components, including `SigLoss`, `DecoderConfig`, and `make_depther_from_config`; it computes RMSE, MAE, AbsRel, log RMSE, SILog, and delta accuracies, uses optional `SyncBatchNorm` and `DDP`, and writes `metrics.json` and `results-depth.csv` `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py` (lines 362–421, 500–505, 572–581, 739–767). The DPT multilayer variant exposes selected V-JEPA block maps to a DINOv3 DPT decoder and changes the default selected layers from `[11, 23, 37, 47]` to `[9, 19, 29, 39]` when the checkpoint has 40 blocks `experiments_related/scripts/07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py` (lines 45–49, 467–538, 618–622, 701–710).

3.3 FiLM, SFP, and altitude-conditioned depth flow

The DPT-FiLM MDE scripts add metadata conditioning to DPT features. The FiLM implementation zero-initializes the affine generator so the initial modulation is the identity `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py` (lines 305–341). The FiLM depth model combines a DINOv3 `FeaturesToDepth` wrapper, a DPT-FiLM head, and a frozen V-JEPA/DINO-compatible backbone wrapper `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py` (lines 344–404). It supports an official scene split and an altitude-generalization split in which training uses 30 m and 50 m from all scenes, validation uses 40 m in scenes 01–04, and testing uses 40 m in scenes 05–08 `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py` (lines 1340–1416).

The altitude-feature variant concatenates an altitude plane to every projected DPT feature level and adapts channels back to the expected DPT dimensions; its checkpoint payload records altitude-injection metadata `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_altitude_feature_occufly.py` (classes `AltitudeInjectedDPTHead` and `VJepaDPTAltitudeFeatureDepth` checkpoint payload around lines 691–710). The SFP variant adds `SimpleFeaturePyramid`, `make_`

`vjepa21_final_sfp_dino_backbone`, and `FinalSFPFiLMDPTHead` before the same FiLM depth decoder training/evaluation loop `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_final_sfp_dpt_film_occufly.py` (classes `SimpleFeaturePyramid`, `FinalSFPFiLMDPTHead`, and training logic).

3.4 Video-depth flow

The video-depth loader first collects sequences from `scene/height/images/visual` and `depth_maps`, attaches calibration paths, and builds target-centered clips. Even-length clips duplicate the target frame in the middle; for example, the code comments define a length-4 offset pattern `[-1, 0, 0, 1]` `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/occufly_data.py` (lines 149–185, 199–244). Clip modes are:

- `repeated_stlp`: target frame repeated through the clip;
- `real_stlp`: temporal context around the target frame, clamped at sequence boundaries;
- `shuffled_stlp`: deterministic shuffled context.

The dataset returns a tensor `[3, T, H, W]`, target depth, valid mask, metadata, UID, scene, height, and target frame fields `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/occufly_data.py` (lines 330–405).

The ST-LP model path is:

OccuFly frame sequence

- > target-centered clip `[B, 3, T, H, W]`
- > frozen V-JEPA2.1 video grid extractor
- > temporal fusion (concat or mean)
- > linear dense depth head
- > `FeaturesToDepth`
- > `SigLoss` or L1 and depth metrics

The video grid extractor returns `[B, C, T', H', W']` and enforces divisibility by tubelet size `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/vjepa_backbone.py` (lines 57–142). The factory constructs a frozen encoder, computes T' , selects concat/mean temporal fusion, and verifies that the optimizer covers exactly trainable ST-LP parameters `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/factory.py` (lines 13–63). The training script writes per-frame metrics, altitude metrics, history, best/latest checkpoints, `results-depth.csv`, and best validation RMSE selection `experiments_related/vjepa21video_scripts/scripts/train_stlp_depth.py` (lines 104–143, 208–351).

The DTA video-depth model uses V-JEPA2.1 video features, dense temporal attention over tubelets at each spatial cell, and a DINOv3 DPT-FiLM depth decoder `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/model.py` (lines 22–112). The temporal adapter uses learned-query cross-attention over the temporal sequence for each spatial location, followed by an MLP and residual mixing coefficient `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/temporal_attention.py` (lines 15–89). Its dataset implements `repeated_dta`, `real_dta`, and `shuffled_dta`; it returns clip images, target depth, valid mask, metadata, target frame ID, and clip frame IDs `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/occufly_video_dataset.py` (lines 147–230, 330–359). The training loop computes depth metrics, best/latest checkpoints, per-altitude and per-frame CSVs, and uses validation RMSE for best-checkpoint selection `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/train_eval.py` (lines 35–138, 162–306).

3.5 SSC data flow

The most complete SSC data flow is implemented in `vjepa21video_posemvs_film_consistent_foundationssc_hybrid`. The dataset module defines train, validation, and test scenes; height

folders; image, depth, and label extensions; and a sample record containing RGB, depth, calibration, semantic label, invalid/occluded masks, clip paths, multi-view source clips, and pose fields `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_ssc_data.py` (lines 25–78). It builds target-centered clip indices and multi-view source indices, with source-view policies for skip or replicate behavior `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_ssc_data.py` (lines 119–200). Pose parsing accepts pickle, NumPy, JSON, and text/table formats; it converts camera-to-world or world-to-camera conventions when needed and raises an error when requested pose evidence is missing `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_ssc_data.py` (lines 253–377).

SSC label loading supports `.label`, `.npy`, `.npz`, `.pt`, and `.pkl`; invalid and occluded masks can be packed or unpacked bytes. The loader maps raw IDs to contiguous train IDs and converts grid orientation to the internal DHW convention `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_ssc_data.py` (lines 732–812). The label map treats raw 0 as empty and maps known raw semantic IDs to contiguous training IDs, with 255 used as the ignore index `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_labels.py` (lines 6–65).

The SSC model path is:

```
OccuFly RGB/video clips + depth + intrinsics + poses + voxel labels
-> V-JEPA image/video context features
-> DPT context feature adapter
-> geometry branch:
    (a) DPT-FiLM depth probability branch, or
    (b) pose-aware MVS depth branch
-> FoundationSSC hybrid view transformer
-> FoundationSSC 3D decoder
-> semantic occupancy logits [B,C,D,H,W]
-> SSC/depth/projection diagnostics and losses
```

The main SSC model contains explicit methods for context features, DPT depth features, pose-MVS view maps, and depth-branch execution `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py` (lines 306–377). Its forward method returns logits plus depth logits/probabilities/dense depth, LSS and voxel volumes, proposals, projection diagnostics, and MVS diagnostics `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py` (lines 379–436).

3.6 CGFormer/Aero-JEPA data flow

The CGFormer/Aero-JEPA dataset file is a metadata-driven adapter. Its docstring states that it expects manifest JSON/JSONL records generated from real OccuFly and lists expected keys such as image path, depth path, SSC label path, valid mask path, intrinsics, poses, altitude, frame ID, scene ID, and timestamp `experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc/data/occufly_dataset.py` (lines 25–35). The same file explicitly states that its placeholder loading logic should be replaced or extended for a concrete storage layout, so it is evidence of a scaffolded adapter rather than a fully self-contained OccuFly reader `experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc/data/occufly_dataset.py` (lines 25–35, 55–144).

The CGFormer training launcher selects either a DAV2-prior backend or a V-JEPA21 ViT-G DPT-FiLM depth-prior backend depending on config flags, writes metadata, exports environment variables, and launches a CGFormer-compatible external training script `experiments_related/`

scripts_ssc/cgformer/aero_jepa_ssc_project/tools/train_occufly.py (lines 1–8, 31–86, 164–301). M2+ variants are documented in launcher metadata as using cached V-JEPA dense teacher features through OCCUFLY_VJEPa_FEATURE_ROOT experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/tools/train_occufly.py (lines 216–258). The ablation queue first caches V-JEPA features and then runs M2–M6 rows before building a table experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/run_ablation_queue.sh (lines 22–57).

4 Model architecture families

4.1 V-JEPA/V-JEPA2.1 usage

The code uses V-JEPA/V-JEPA2.1 in three distinct ways:

1. **Frozen feature extraction for depth.** MDE scripts freeze the encoder and train a linear head or DPT decoder. The linear script asserts encoder freezing and optimizer membership exactly over head parameters experiments_related/scripts/03_train_vjepa_linear_depth.py (lines 474–493). The DPT scripts assert that only decoder parameters are optimized experiments_related/scripts/07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py (decoder-freezing and optimizer checks around training setup).
2. **Self-supervised OccuFly adaptation.** The RGB finetuning script writes V-JEPA2.1 configs with lambda_value_img=1.0 and lambda_value_vid=0.0; the video finetuning script writes configs with lambda_value_img=0.0 and lambda_value_vid=1.0 experiments_related/scripts/08_vjepa21_finetune_occufly.py (lines 187–293) experiments_related/scripts/11_vjepa21_finetune_occufly_video.py (lines 251–357).
3. **Video-grid extraction for temporal depth and SSC.** The video-grid extractor adapts V-JEPA2.1 to return $[B, C, T', H', W']$ selected-layer grids for clip inputs experiments_related/vjepa21video_scripts/src/vjepa21video_depth/vjepa_backbone.py (lines 57–142). The SSC model uses video context features and, for pose MVS, per-view video features before multi-view correlation experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationsssc_hybrid/vjepaafoundationsssc/train_vjepa_occufly_ssc.py (lines 306–377).

External grounding: V-JEPA is a joint-embedding predictive video representation method [1]; V-JEPA 2 is the successor self-supervised video model family referenced by the code’s facebookresearch/vjepa2 loaders [2, 3].

4.2 DINOv3, DPT, and V-JEPA-to-DINO adapters

The adapter module vjepa21_dinov3_depth_adapter.py defines how a V-JEPA2.1 encoder is exposed through a DINOv3 backbone contract. It recognizes checkpoint keys such as ema_encoder, target_encoder, and encoder; infers V-JEPA size from embedding dimension; constructs V-JEPA2.1 base/large/giant/gigantic backbones; and returns dummy CLS tokens plus reshaped patch maps for DINOv3 depth heads experiments_related/scripts/vjepa21_dinov3_depth_adapter.py (lines 55–145, 148–234). The multilayer wrapper exposes selected intermediate block maps and adjusts default DPT layer indices for 40-block V-JEPA-g checkpoints experiments_related/scripts/vjepa21_dinov3_depth_adapter.py (lines 237–363). This connects V-JEPA features to DPT-style dense prediction, whose external architectural reference is DPT [6]. The DINOv3 dependency is grounded by the code imports from dinov3.eval.depth and the external DINOv3 implementation/report [4, 5].

4.3 Linear, DPT, FiLM, SFP, and probability depth heads

The implemented depth-head families are:

- **Dense linear depth head:** a 1x1 convolution on frozen V-JEPA patch-grid features experiments_related/scripts/03_train_vjepa_linear_depth.py (lines 260–284).

- **DINOv3 linear depth probe:** DINOv3-style depther decoder on frozen V-JEPA features, optimized by DINOv3 SigLoss experiments `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py` (lines 362–366, 500–505, 576–581).
- **Multilayer DPT decoder:** selected V-JEPA block maps are routed to a DINOv3 DPT decoder experiments `experiments_related/scripts/07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py` (lines 45–49, 618–622).
- **DPT-FiLM decoder:** DPT feature maps are modulated by an altitude/intrinsics embedding before fusion/refinement experiments `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py` (lines 305–404).
- **Altitude feature-plane DPT:** altitude is concatenated as a spatial plane to DPT feature levels before channel adaptation experiments `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_altitude_feature_occufly.py` (classes `AltitudeInjectedDPTHead` and `VJepaDPTAltitudeFeature`).
- **Final-layer SFP + DPT-FiLM:** a simple feature pyramid is derived from final V-JEPA features before DPT-FiLM decoding experiments `experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_final_sfp_dpt_film_occufly.py` (classes `SimpleFeaturePyramid` and `FinalSFPFiLMDPTHead`).
- **Depth probability head for SSC:** DPT-FiLM or pose-MVS geometry branches output depth logits/probabilities over metric bins and dense depth as expectation, so SSC view transforms receive both probabilities and context features experiments `experiments_related/scripts/ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/pose_mvs_geometry.py` (lines 156–205, 208–298).

4.4 Temporal attention and video adaptation

ST-LP implements two fusion operators: temporal concatenation of $[B, C, T, H, W]$ into $[B, TC, H, W]$, and temporal mean pooling experiments `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/depth_heads.py` (lines 39–76). DTA implements dense temporal attention using a learned query over tubelets at every spatial cell experiments `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/temporal_attention.py` (lines 15–89). In the SSC model, separate temporal adapters can be used for context and depth branches before FoundationSSC hybrid view transformation experiments `experiments_related/scripts/ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py` (lines 730–793).

4.5 Semantic scene completion architectures

The FoundationSSC hybrid implementation rewrites FoundationSSC geometry configuration to the OccuFly grid, checks for stale SemanticKITTI dimensions, rebins depth probabilities to LSS bins, runs FoundationSSC LSS/proposal/VoxFormer/dual-fusion modules, and decodes to DHW semantic logits experiments `experiments_related/scripts/ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/foundation_ssc/real_adapters.py` (lines 44–64, 103–141, 169–229, 326–456, 459–499). The code imports and bootstraps FoundationSSC rather than reimplementing it fully; FoundationSSC itself is externally described as a semantic/geometric prior decoupling framework [9, 10]. The use of VoxFormer-like modules is explicit through the FoundationSSC adapter’s `VoxFormer` and hybrid view transformer calls experiments `experiments_related/scripts/ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/foundation_ssc/real_adapters.py` (lines 397–456); VoxFormer is externally grounded by the CVPR 2023 paper [11].

The pose-MVS branch replaces a frozen monocular DPT geometry branch with multi-view pose-aware correlation. It projects target depth hypotheses into source views using intrinsics and world-to-camera transforms, samples source view features with `grid_sample`, aggregates cosine correlations, regularizes the cost volume with a 3D CNN, and conditions output depth logits with metadata FiLM experiments `experiments_related/scripts/ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/pose_mvs_geometry.py` (lines 208–360). The branch returns a dense depth expectation consistent with its conditioned probability distribution experiments

related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc_posemvs_geometry.py (lines 156–205, 260–298).

4.6 CGFormer / Aero-JEPA architecture variants

The Aero-JEPA project provides a CGFormer-compatible wrapper rather than a standalone CGFormer implementation. The orchestrator runs a base CGFormer-compatible model, adds altitude-lifting outputs where configured, requires depth teachers for in-model frustum variants during training, and optionally runs a V-JEPA inference branch under `torch.no_grad()` `experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc/models/aero_jepa_ssc.py` (lines 13–97). Altitude lifting includes a depth-head variant and a probabilistic frustum splatter that maps per-pixel feature/depth distributions into a voxel volume `experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc/models/altitude_lifting.py` (lines 14–134, 137–243). CGFormer is externally grounded by the context-and-geometry-aware voxel transformer paper [8].

5 Experiment variants and ablations

5.1 MDE and video-depth variants

ID	Entry / launch evidence	Inputs and model	Losses / metrics	Implemented test
MDE-LP	<code>experiments_related/scripts/03_train_vjepa_linear_depth.py</code> ; standard CLI entrypoint inferred from argument parser and training function	OccuFly RGB/depth; frozen V-JEPA2.1; 1x1 dense linear head	Masked MSE; RMSE; best validation RMSE <code>experiments_related/scripts/03_train_vjepa_linear_depth.py</code> (lines 425–455, 636–670)	Whether frozen V-JEPA2.1 patch features linearly predict depth.
MDE-DINO-LP	<code>experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py</code>	OccuFly RGB/depth; V-JEPA backbone adapted to DINOv3 depther decoder	Masked SigLoss; RMSE, MAE, AbsRel, log RMSE, SILog, delta1–3 <code>experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py</code> (lines 362–421)	Whether DINOv3-style depth probing improves over a plain linear head.

ID	Entry / launch evidence	Inputs and model	Losses / metrics	Implemented test
MDE-DPT	experiments_related/scripts/07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py	Selected V-JEPA layers to DPT decoder; frozen encoder	SigLoss and the same depth metrics; SyncBN/DDP optional experiments_related/scripts/07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py (lines 467–538, 701–710)	Whether multi-level V-JEPA features support DPT-style dense decoding.
MDE-DPT-FiLM	experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py	OccuFly RGB/depth/calibration/altitude DPT V-JEPA-DPT features; metadata FiLM	DINOv3 SigLoss; optional depth-bin/probability settings; per-altitude metrics and evaluation/finalization args experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py (lines 305–450, 964–1040, 1340–1416)	Whether altitude and intrinsic conditioning improve/alter monocular depth estimation.
MDE-alt-plane	train_vjepa21_vitG_dpt_altitude_feature_occufly.py under scripts/dpt_decoder	Altitude plane concatenated to projected DPT levels	Same DPT-FiLM family metrics; checkpoint records altitude injection metadata	Alternative conditioning mechanism: spatial altitude feature vs FiLM.

ID	Entry / launch evidence	Inputs and model	Losses / metrics	Implemented test
MDE-SFP-FiLM	<code>train_vjepa21_vitG_final_sfp_dpt_film_occufly.py</code> under <code>scripts/dpt_decoder</code>	Final V-JEPA layer plus simple feature pyramid feeding DPT-FiLM head	Same depth metrics; debug prints feature shapes and best <code>dpt_film_best.pt</code> checkpoint	Whether final-layer SFP can substitute for explicit multi-layer V-JEPA features.
RGB-JEPA-FT	<code>experiments_related/scripts/08_vjepa21_finetune_occufly.py</code> ; launches <code>vjepa2/app/main.py</code>	RGB frames only, one-frame clips; V-JEPA2.1 self-supervised objective	JEPA image loss; no depth/label loss <code>experiments_related/scripts/08_vjepa21_finetune_occufly.py</code> (lines 2–9, 187–360)	OccuFly RGB-domain adaptation of V-JEPA2.1 without supervised depth labels.
Video-JEPA-FT	<code>experiments_related/scripts/11_vjepa21_finetune_occufly_video.py</code> ; launches <code>vjepa2/app/main.py</code>	Materialized MP4 videos from OccuFly frame sequences	JEPA video loss; no depth/label loss <code>experiments_related/scripts/11_vjepa21_finetune_occufly_video.py</code> (lines 176–240, 251–427)	OccuFly temporal adaptation of V-JEPA2.1.
FT-probe wrappers	<code>experiments_related/scripts/10_vjepa21_finetuned_linearprobe_occufly.py</code> ; <code>11_vjepa21_finetuned_DPT_multilayer_decoder_occufly.py</code>	Fine-tuned RGB/video V-JEPA checkpoints selected from output dirs	Reuses base LP/DPT training and metrics	Whether OccuFly-adapted V-JEPA features improve depth probes.

ID	Entry / launch evidence	Inputs and model	Losses / metrics	Implemented test
ST-LP repeated/real/shuffled	experiments_related/vjepa21video_scripts/scripts/train_stlp_depth.py	Target-centered clips; frozen V-JEPA video grid; concat or mean temporal fusion; linear head	SigLoss or L1; depth metrics and per-frame/per-altitude CSVs experiments_related/vjepa21video_scripts/scripts/train_stlp_depth.py (lines 104–143, 208–351)	Whether real temporal context helps relative to repeated or shuffled context.
DTA-DPT-FiLM repeated/real/shuffled	vjepa21video_dptfilm_attention_scripts/train_video_dptfilm_attention.py	V-JEPA video features; dense temporal attention; DINOv3 DPT-FiLM decoder	SigLoss; same depth metrics; best dta_dptfilm_best.pt experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/train_eval.py (lines 35–138, 162–306)	Whether learned temporal attention helps video depth prediction.
Constant-depth / diagnostics	experiments_related/scripts/05_constant_depth_baseline.py; scripts/diagnostics/*.py	OccuFly depth maps and predictions where applicable	Baseline depth errors, scale-error and prediction-shift diagnostics	Sanity checks and baseline calibration rather than a trainable architecture.

5.2 SSC variants

ID	Entry / launch evidence	Inputs and model	Losses / metrics	Implemented test
Simple V-JEPA-SSC	experiments_related/scripts_ssc/vjepafoundationssc	V-JEPA multi-layer backbone, context head, DPT-FiLM depth-probability head, frustum voxel pooling, simple 3D decoder	Local SSC/depth losses and metrics in the scaffold	Early end-to-end V-JEPA-to-SSC path.
FoundationSSC hybrid DPT-depth	vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py with geometry_branch=dpt_depth	RGB or video V-JEPA context; DPT-FiLM depth probabilities; FoundationSSC hybrid view transformer and decoder	Total loss combines SSC semantic/scaling/occupancy and depth losses; metrics include ssc_miou, occ_iou/sc_iou, depth diagnostics experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py (lines 680–878, 888–972)	Whether pre-trained/frozen depth priors can drive FoundationSSC-style SSC on OccuFly.

ID	Entry / launch evidence	Inputs and model	Losses / metrics	Implemented test
FoundationSSC pose-MVS	Same path with <code>geometry_branch=pose_mvs</code> ; pose MVS requires <code>input_mode=video</code>	Multi-view RGB/video clips, intrinsics, poses, V-JEPA view features, pose-aware cost volume, FiLM-conditioned depth probabilities, FoundationSSC view transformer	Same SSC/depth losses; MVS diagnostics include valid-warp fraction and source-view counts <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/pose_mvs_geometry.py</code> (lines 208–360) <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py</code> (lines 342–377, 1457–1581)	Whether pose-aware multi-view geometry replaces monocular DPT depth in SSC.
FoundationSSC RGB-current depth source	Same SSC path with <code>dpt_depth_feature_source=rgb_current</code>	Optional RGB-current depth backbone for depth branch instead of video DTA feature source	Same loss and metric stack	Ablation of depth feature source.
SSC de-bug/eval/visual modes	Same SSC train script; visualization scripts in sibling dirs	Existing model/checkpoint inputs, batch data, voxel outputs	Shape assertions; metrics JSON/CSV; live/static viewers	Diagnostics for shape, probability, metric, and visualization consistency.

5.3 CGFormer/Aero-JEPA Table 3 variants

The CGFormer/Aero-JEPA configs implement the following ablation rows. This table records code/config evidence only; it does not report measured results.

Row	Config evidence	Enabled mechanisms	Implemented test
Baseline	<code>configs/occufly/baseline.yaml</code> and <code>tools/run_table3_experiments.sh</code>	No altitude lifting, V-JEPA distillation, latent view, masked 3D, or output consistency losses.	CGFormer-compatible OccuFly baseline.
M1 prior	<code>m1_prior_altitude_lifting.yaml</code>	Altitude-conditioned depth-prior backend; depth distribution and altitude-normalized depth loss.	Altitude/depth prior without in-model GT/V-JEPA teacher branch.
M1 real GT	<code>m1_real_gtdepth.yaml</code>	In-model probabilistic frustum lifting using GT metric-depth teacher; teacher disabled at inference.	GT-depth teacher variant.
M1 real VJ	<code>m1_real_vjepateacher.yaml</code>	In-model probabilistic frustum lifting using V-JEPA21 ViT-G DPT-FiLM depth-prior teacher; distill-only mode; FiLM conditioning by altitude/camera intrinsics.	V-JEPA depth-teacher variant.
M1 real GT+VJ	<code>m1_real_gtdepth_vjepateacher.yaml</code>	Both GT-depth and V-JEPA teacher losses with separate KL/L1 weights.	Combined teacher variant.
M2	<code>m2_vjepa_distill.yaml</code>	Altitude backend plus V-JEPA distillation loss.	2D V-JEPA teacher feature distillation.
M3	<code>m3_latent_view.yaml</code> ; <code>m3_no_2d_distill_latent_view_only.yaml</code>	Paired training with V-JEPA distillation and latent-view consistency, or latent-only ablation.	Whether latent view consistency contributes independently or with 2D distillation.
M4	<code>m4_masked3d_*.yaml</code>	Masked-3D loss with block, column, frustum, mixed, or random masks.	Mask pattern ablation.
M5	<code>m5_latent_view_masked3d.yaml</code>	V-JEPA distillation, latent-view consistency, and masked-3D loss.	Combined M2/M3/M4 mechanisms.
M6	<code>m6_full_with_output_consistency.yaml</code>	M5 plus output-consistency loss.	Full ablation stack.

Row	Config evidence	Enabled mechanisms	Implemented test
Table generator	<code>tools/make_table3.py</code>	Row metadata verification against expected flags; writes CSV, Markdown, and TeX tables <code>experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/tools/make_table3.py</code> (lines 19–184).	Prevents table rows from being built from mismatched config metadata.

6 Losses, metrics, optimization, and checkpointing

6.1 Depth losses and metrics

Name	Definition in code	Evidence
Masked MSE	Mean squared error over valid depth pixels.	<code>masked_mse</code> in <code>experiments_related/scripts/03_train_vjepa_linear_depth.py</code> (lines 425–455).
DINOv3 SigLoss	Imported DINOv3 <code>SigLoss</code> , applied with valid masks in linear/DPT/DTA variants.	<code>experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py</code> (lines 362–366, 500–505); <code>experiments_related/scripts/07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py</code> (lines 618–622); <code>experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/train_eval.py</code> (lines 35–39).
Masked L1 / relative L1	Video ST-LP can use L1; SSC depth L1 supports meter or relative mode.	<code>experiments_related/vjepa21video_scripts/src/vjepa21video_depth/losses_metrics.py</code> (classes <code>MaskedL1Loss</code> and <code>MetricAccumulator</code>); <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/losses_ssc.py</code> (lines 89–107).

Name	Definition in code	Evidence
Hard depth CE	Metric GT depth is assigned to depth-bin indices and supervised by cross entropy.	<code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/losses_ssc.py</code> (lines 35–48).
Soft depth KL/CE	Gaussian target distributions around metric GT depth supervise predicted bin distributions.	<code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/losses_ssc.py</code> (lines 51–86).
Depth probability loss	Combines soft/hard CE and L1 terms under configurable weights.	<code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/losses_ssc.py</code> (lines 110–138).
Altitude depth distribution/loss	CGFormer/Aero-JEPA altitude depth loss uses CE over metric/altitude-normalized bins, expected-depth L1, and optional smoothness.	<code>experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc/models/altitude_lifting.py</code> (lines 246–323).

Depth metrics are implemented as follows. RMSE is $\sqrt{\text{mean}((\hat{d} - d)^2)}$ over valid pixels; MAE is mean absolute error; AbsRel is $\text{mean}(|\hat{d} - d|/d)$; SqRel is $\text{mean}((\hat{d} - d)^2/d)$; log RMSE and SILog are computed from valid log-depth differences; delta accuracies count valid pixels satisfying $\max(\hat{d}/d, d/\hat{d}) < 1.25^k$ for $k \in \{1, 2, 3\}$. These accumulators appear in the DINOv3-style probe, DPT, ST-LP, and DTA video-depth paths `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py` (lines 370–421) `experiments_related/scripts/07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py` (lines 467–538) `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/losses_metrics.py` (lines 69–143) `experiments_related/vjepa21video_scripts/vjepa21video_dptfilm_attention/src/vjepa21video_dptfilm_attention/train_eval.py` (lines 42–99).

6.2 SSC losses and metrics

The FoundationSSC hybrid loss stack includes semantic cross-entropy over voxel logits, geometric scaling loss, semantic scaling loss, occupancy loss from semantic logits, and optional depth CE/L1 terms. The implementation exposes a `total_loss` that combines these terms with user-provided weights `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/losses_ssc.py` (lines 141–172 and total-loss function around line 221). The SSC training loop calls `total_loss` on model outputs and batch labels/depth maps during training `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py` (lines 1271–1293).

The SSC metric accumulator treats raw/train class 0 as empty/free and 255 as invalid/unknown.

It includes empty voxels in IoU unions so false positives in free space are penalized, while excluding class 0 from the final semantic mIoU; this behavior is stated in the module docstring and tested in `test_ssc_metric_accumulator.py` `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/metrics_ssc.py` (lines 14–22, 47–116). The metrics include `sc_iou`, `occ_iou`, `ssc_miou`, `miou_occupied_classes`, `miou_with_empty`, per-class IoU, valid voxel fraction, and occupied voxel fraction `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/metrics_ssc.py` (lines 82–137).

CGFormer/Aero-JEPA losses are dispatched by `AeroJEPASSCLoss`. The loss adds baseline SSC loss and optional depth, GT/V-JEPA teacher, V-JEPA distillation, latent-view, masked-3D, and output-consistency losses according to flags and weights `experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc/losses/aero_jepa_ssc_loss.py` (lines 13–117). Table 3 provenance checking verifies that run metadata matches each expected row before table generation `experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/tools/make_table3.py` (lines 19–184).

6.3 Optimization, DDP, schedulers, and checkpoint selection

The code consistently separates frozen representation parameters from trainable heads/decoders. Examples include strict linear-probe checks in MDE `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 474–493), decoder-only optimization in DINOv3 linear/DPT probes `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py` (lines 576–581), ST-LP trainable-parameter checks `experiments_related/vjepa21video_scripts/src/vjepa21video_depth/factory.py` (lines 50–63), and SSC optimizer-group coverage checks across depth, context, hybrid view, and decoder groups `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py` (lines 888–925).

Distributed training is implemented in several paths. The DINOv3-style probe reads `WORLD_SIZE`, `RANK`, and `LOCAL_RANK`, uses NCCL for CUDA, and supports a timeout environment variable `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py` (lines 99–119). Depth scripts optionally convert to `SyncBatchNorm` before DDP wrapping `experiments_related/scripts/06_vjepa21_dinov3_linearprobe_occufly.py` (lines 572–581). The SSC script includes DDP and AMP helpers, gradient-scaler use, non-finite loss/gradient skips, and warmup+cosine LambdaLR scheduling `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py` (lines 90–122, 888–925, 1171–1454).

Best-checkpoint rules differ by task. Depth scripts select by validation RMSE `experiments_related/scripts/03_train_vjepa_linear_depth.py` (lines 636–670) `experiments_related/vjepa21video_scripts/scripts/train_stlp_depth.py` (lines 208–284). The FoundationSSC hybrid script selects the best checkpoint lexicographically by (`ssc_miou`, `sc_iou/occ_iou`, `-depth_rmse`) and then loads the best checkpoint for final test evaluation `experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py` (lines 1384–1447).

7 Implementation connections and dependency graph

7.1 Explicit and inferred stage connections

Producer	Consumer	Evidence and status
V-JEPA2.1 public/local checkpoint	MDE linear/DINO/DPT/FiLM/video/SSC scripts	Explicit checkpoint defaults and loaders in MDE and SSC scripts <code>experiments_related/scripts/03_train_vjepa_linear_depth.py</code> (lines 38–45, 320–352); <code>experiments_related/scripts/vjepa21_dinov3_depth_adapter.py</code> (lines 55–145).
RGB/video OccuFly V-JEPA finetune outputs	Fine-tuned linear/DPT wrappers	Explicit wrapper scripts select RGB or video finetuned checkpoint dirs and call base depth scripts <code>experiments_related/scripts/11_vjepa21_finetuned_DPT_multilayer_decoder_occufly.py</code> (lines 2–12, 27–128, 172–204).
MDE DPT-FiLM best checkpoint	SSC DPT-depth branch	Explicit <code>DEFAULT_MDE_CHECKPOINT</code> ; SSC build function can load MDE checkpoint into frozen depth head for <code>geometry_branch=dpt_depth</code> <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py</code> (lines 73–76, 846–857).
Video DTA features	SSC video context/depth branches	Explicit construction of video backbones and context/depth temporal attention adapters in SSC build function <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py</code> (lines 730–793).

Producer	Consumer	Evidence and status
Pose and MVS source clips	Pose-aware MVS depth branch	Explicit dataset return fields and model method <code>_extract_pose_mvs_view_maps;</code> pose-MVS branch requires video input and multi-view clips <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_ssc_data.py</code> (lines 172–200, 253–377, 814–886); <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py</code> (lines 342–377).
V-JEPA depth-prior tree / cached features	CGFormer M1/M2+ ablations	Explicit environment variables and metadata in CGFormer launcher; ablation queue runs feature caching before M2–M6 <code>experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/tools/train_occufly.py</code> (lines 216–258); <code>experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/run_ablation_queue.sh</code> (lines 22–57).
Segmentation label map	OccuFly segmentation linear sweep	Inference from label-map path and shell launcher; the export contains only a label JSON and wrappers that exec DINOv3 tools, not the implementation of those external tools.

7.2 Text dependency graph

External repos/checkpoints:

`facebookresearch/vjepa2` or local `vjepa2`
`dinov3/eval/depth`
FoundationSSC external tree
CGFormer/Efficient-Swin-OccuFly external tree
optional DAV2 / V-JEPA depth-prior trees

OccuFly dataset:

RGB frames + depth maps + calibration
+ semantic voxel labels + invalid/occluded masks
+ optional poses and MVS source frames

MDE path:

OccuFly RGB/depth -> frozen V-JEPA -> LP/DPT/DPT-FiLM/SFP -> depth metrics

Video depth path:

OccuFly sequence -> target-centered clip -> V-JEPA video grids
-> ST-LP or dense temporal attention -> depth metrics

SSC path:

OccuFly SSC sample -> V-JEPA context + geometry depth branch
-> FoundationSSC hybrid view transform -> FoundationSSC decoder
-> SSC/depth metrics

CGFormer ablation path:

OccuFly metadata + depth-prior/cache roots -> CGFormer backend launcher
-> row metadata -> Table 3 generator

7.3 Shared conventions

Shared filesystem conventions include hard-coded default roots under `/datatank/giridhar.vb/...`, fixed scene splits, height folders 30, 40, and 50, V-JEPA2.1 checkpoint names such as `vjepa2_1_vitG_384.pt`, and output names such as `metrics.json`, `results-depth.csv`, `history.csv`, `*_best.pt`, and `*_latest.pt`. Shared metadata conventions include the altitude/intrinsics vector used by FiLM and SSC depth branches. Shared DPT conventions include selected layers, DPT feature dimensions, depth-bin/norm choices, and DINOv3 `FeaturesToDepth` wrappers.

8 Scientific completeness, constraints, and reproducibility risks

The following items are factual constraints or failure modes visible in code; they are not claims about actual experiment outcomes.

Finding	Evidence
No numeric result validation from the export	Outputs, checkpoints, logs, generated predictions, and large tensors are excluded. The code writes metrics and checkpoints, but the archive does not contain run artifacts to verify any reported score.
External repository dependency	MDE adapters depend on local or remote V-JEPA2 and DINOv3 modules <code>experiments_related/scripts/vjepa21_dinov3_depth_adapter.py</code> (lines 462–473). FoundationSSC adapters bootstrap an external FoundationSSC tree and import <code>mmdet3d_plugin experiments_related/scripts/ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/foundationssc/real_adapters.py</code> (lines 44–64). CGFormer launchers select external backend scripts <code>experiments_related/scripts/ssc/cgformer/aero_jepa_ssc_project/tools/train_occufly.py</code> (lines 73–86, 279–301).

Finding	Evidence
Hard-coded local path assumptions	Default dataset/checkpoint/output roots point to /datatank/giridhar.vb/... in MDE, video-depth, and CGFormer scripts <code>experiments_related/scripts/03_train_vjepa_linear_depth.py</code> (lines 38–45); <code>experiments_related/vjepa21video_scripts/src/vjepa21video_depth/constants.py</code> (lines 7–22); <code>experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/tools/train_occufly.py</code> (lines 31–44).
Pose-MVS input preconditions	Pose MVS requires video input, multi-view clips, intrinsics, and world-to-camera transforms; pose lookup raises when requested pose data is missing <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/train_vjepa_occufly_ssc.py</code> (lines 232–264, 342–377); <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_ssc_data.py</code> (lines 253–377).
Grid and coordinate convention sensitivity	The SSC loader converts labels to internal DHW orientation, remaps raw IDs, validates shapes, and the FoundationSSC adapter errors if SemanticKITTI default dimensions remain after rewriting <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/occufly_ssc_data.py</code> (lines 794–812); <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/foundation_ssc/real_adapters.py</code> (lines 169–229).
Metric semantics must be reported carefully	The SSC metric includes empty/free voxels in IoU unions to penalize free-space false positives, but excludes class 0 from final semantic mIoU <code>experiments_related/scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc/metrics_ssc.py</code> (lines 14–22, 82–116); corresponding tests exist in <code>experiments_related/tests/test_ssc_metric_accumulator.py</code> .
CGFormer metadata adapter is not a complete raw-reader by itself	The Aero-JEPA OccuFly dataset docstring states that it expects real manifests and that placeholder loading should be replaced or extended for concrete storage <code>experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc/data/occufly_dataset.py</code> (lines 25–35).
CGFormer in-model frustum variants require patched hooks	The CGFormer launcher detects real in-model CGFormer hooks and refuses to silently fall back when a required hook is absent <code>experiments_related/scripts_ssc/cgformer/aero_jepa_ssc_project/tools/train_occufly.py</code> (lines 53–70 and in-model fallback checks near launch).

Finding	Evidence
Split-protocol ambiguity is explicit in code	The MDE DPT-FiLM script supports both an official scene split and an altitude split that trains on 30/50 m and evaluates 40 m; those protocols answer different questions and should not be merged in paper tables without separate labels <code>experiments_related/scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py</code> (lines 1340–1416).
Duplicate/mirror content	The archive includes <code>forpaper_report</code> snapshots in addition to primary experiment code. This review treats mirrors as supporting copies, not independent implementations.

9 Paper-facing summaries

9.1 Concise implementation summary

The implemented system studies OccuFly monocular/video depth and semantic scene completion using V-JEPA/V-JEPA2.1 representations. The depth side includes frozen V-JEPA linear probes, DINOv3-style depth probing, DPT multilayer decoding, altitude/intrinsics FiLM conditioning, SFP final-layer variants, RGB/video self-supervised V-JEPA adaptation, and video temporal-ablation probes. The SSC side contains a FoundationSSC-hybrid pipeline in which V-JEPA RGB/video context features and either DPT-FiLM depth probabilities or pose-aware multi-view stereo depth probabilities are transformed into 3D volumes and decoded into semantic occupancy. A separate CGFormer/Aero-JEPA project implements ablation configurations for altitude priors, V-JEPA teachers, latent-view consistency, masked-3D learning, and output consistency. This summary is supported by the primary paths and evidence cited in Sections 2–5; it is not a claim of completed numerical performance.

9.2 Nomenclature and symbols

Term / symbol	Meaning in the implementation
V-JEPA / V-JEPA2.1	Video/image representation backbone loaded from V-JEPA/V-JEPA2 repositories or checkpoints and used frozen or self-supervised finetuned. External references: [1, 2].
DINOv3 depther	DINOv3 depth-evaluation components such as <code>SigLoss</code> , <code>DecoderConfig</code> , <code>FeaturesToDepth</code> , and DPT heads used with V-JEPA feature adapters [4, 5].
DPT	Dense Prediction Transformer style multilayer dense prediction decoder [6].
FiLM	Feature-wise linear modulation by metadata embedding, implemented for altitude and camera intrinsics; affine generators are initialized to identity in the inspected FiLM blocks.
SFP	Simple Feature Pyramid derived from final V-JEPA features before DPT-FiLM decoding.
ST-LP	Spatio-temporal linear probe: V-JEPA video grid features fused by temporal concat or mean, then a linear depth head.
DTA	Dense Temporal Attention: learned-query temporal attention over tubelets at each spatial cell before DPT-FiLM decoding.
SSC	Semantic Scene Completion: dense voxel occupancy and semantic prediction.
SC IoU / Occ IoU	Completion/occupancy IoU computed by collapsing non-empty classes to occupied.

Term / symbol	Meaning in the implementation
SSC mIoU	Mean IoU over semantic classes excluding empty class 0 but respecting ignore label 255.
$d, \hat{d}$	Ground-truth and predicted depth.
K	Camera intrinsics matrix; per-sample metadata also uses normalized f_x, f_y, c_x, c_y .
T_{wc}, T_{cw}	World-to-camera and camera-to-world transforms as parsed/converted by pose-loading code.
D, H, W	Internal SSC voxel dimensions in depth-height-width convention after label conversion.

9.3 Text diagrams

Depth pipeline.

OccuFly RGB/depth/calibration

- > mask-aware resize + valid mask + metadata
- > V-JEPA2.1 frozen/adapted backbone
- > {linear head | DINOv3 depther | DPT | DPT-FiLM | SFP-DPT-FiLM}
- > masked depth loss
- > RMSE / MAE / AbsRel / SqRel / SILog / delta metrics

Video depth pipeline.

OccuFly ordered frames

- > repeated / real / shuffled target-centered clips
- > V-JEPA2.1 video grids
- > {temporal concat/mean ST-LP | dense temporal attention}
- > depth decoder
- > per-frame and per-altitude depth metrics

FoundationSSC hybrid pipeline.

OccuFly RGB/video + intrinsics + depth + SSC labels + optional poses

- > V-JEPA context features
- > context DPT/F2D adapter
- > geometry branch:
 - DPT-FiLM depth probabilities OR pose-aware MVS depth probabilities
- > FoundationSSC hybrid view transformer
- > FoundationSSC 3D decoder
- > semantic occupancy logits
- > SSC loss + depth loss + SSC/depth diagnostics

Experiment-family tree.

experiments_related

```

|-- scripts
|   |-- MDE: linear, DINOv3 linear, DPT, DPT-FiLM, SFP, baselines
|   |-- V-JEPA2.1 RGB/video finetuning wrappers
|   '-- diagnostics and visualizations
|-- vjepa21video_scripts
|   |-- ST-LP video depth
|   '-- DTA + DPT-FiLM video depth

```

```

|-- scripts_ssc
|   |-- vjepafoundationssc scaffold
|   |-- FoundationSSC hybrid variants
|   |-- pose-MVS + FiLM-consistent FoundationSSC hybrid
|   '-- cgformer/aero_jepa_ssc_project Table 3 ablations
|-- scripts_segmentation
|   '-- VOC12/OccuFly linear sweep wrappers and label map
'-- tests
    '-- mask-aware depth resize, SSC metrics, Aero-JEPA components

```

9.4 What the implementation can support in a paper

- **Methods section support.** The code supports a methods description covering frozen V-JEPA/V-JEPA2.1 depth probing, DINOv3/DPT decoding, metadata FiLM, video temporal attention, pose-aware MVS depth probabilities, and FoundationSSC hybrid view transformation. The supporting code is cited in Sections 3–4.
- **Ablation section support.** The code supports ablations for linear vs DPT depth heads, multi-layer vs SFP features, FiLM/altitude conditioning, RGB/video V-JEPA adaptation, repeated vs real vs shuffled temporal context, DPT-depth vs pose-MVS SSC geometry, RGB-current vs video-DTA depth features, and CGFormer M1–M6 Table 3 mechanisms. The supporting entry points are listed in Section 5.
- **Limitations section support.** The implementation evidence supports limitations around external dependency on V-JEPA2/DINOv3/FoundationSSC/CGFormer trees, hard-coded local paths, lack of included outputs for result verification, coordinate/grid-shape sensitivity, pose availability for MVS, and explicit split-protocol differences. These are factual code-level constraints listed in Section 8.

10 Appendix: concise source checklist

- MDE core: 03_train_vjepa_linear_depth.py; 06_vjepa21_dinov3_linearprobe_occufly.py; 07_vjepa21_dinov3_DPT_multilayer_decoder_occufly.py; vjepa21_dinov3_depth_adapter.py.
- MDE FiLM/SFP/altitude: scripts/dpt_decoder/train_vjepa21_vitG_dpt_film_occufly.py; train_vjepa21_vitG_dpt_altitude_feature_occufly.py; train_vjepa21_vitG_final_sfp_dpt_film_occufly.py.
- V-JEPA adaptation: 08_vjepa21_finetune_occufly.py; 11_vjepa21_finetune_occufly_video.py; fine-tuned depth wrappers.
- Video depth: vjepa21video_scripts/src/vjepa21video_depth; vjepa21video_dptfilm_attention/src.
- SSC hybrid: scripts_ssc/vjepa21video_posemvs_film_consistent_foundationssc_hybrid/vjepafoundationssc.
- SSC data/labels/loss/metrics: occufly_ssc_data.py; occufly_labels.py; losses_ssc.py; metrics_ssc.py in the same hybrid directory.
- FoundationSSC adapters: foundation_ssc/real_adapters.py.
- Pose MVS: pose_mvs_geometry.py.
- CGFormer/Aero-JEPA: scripts_ssc/cgformer/aero_jepa_ssc_project/src/aero_jepa_ssc; configs under configs/occufly; launch/table scripts under tools.
- Segmentation: scripts_segmentation/dataset_download/download_voc12.sh; scripts_segmentation/linear_pipeline/run_vjepa21_occufly_linear_sweep.sh; run_vjepa21_voc12_linear_sweep.sh; occufly_21class_label_map.json.
- Tests: experiments_related/tests; scripts_ssc/cgformer/aero_jepa_ssc_project/tests.

References

- [1] Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. *Revisiting Feature Prediction for Learning Visual Representations from Video*. arXiv:2404.08471, 2024. <https://arxiv.org/abs/2404.08471>.
- [2] Mido Assran et al. *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*. arXiv:2506.09985, 2025. <https://arxiv.org/abs/2506.09985>.
- [3] Meta AI Research. *facebookresearch/vjepa2: V-JEPA 2 reference implementation*. GitHub repository. <https://github.com/facebookresearch/vjepa2>.
- [4] Meta AI Research. *DINOv3*. arXiv:2508.10104, 2025. <https://arxiv.org/abs/2508.10104>.
- [5] Meta AI Research. *facebookresearch/dinov3: reference PyTorch implementation and models*. GitHub repository. <https://github.com/facebookresearch/dinov3>.
- [6] Rene Ranftl, Alexey Bochkovskiy, and Vladlen Koltun. *Vision Transformers for Dense Prediction*. ICCV 2021; arXiv:2103.13413. <https://arxiv.org/abs/2103.13413>.
- [7] Markus Gross, Sai B. Matha, Aya Fahmy, Rui Song, Daniel Cremers, and Henri Meess. *OccuFly: A 3D Vision Benchmark for Semantic Scene Completion from the Aerial Perspective*. CVPR 2026; arXiv:2512.20770. <https://arxiv.org/abs/2512.20770>; project page <https://markus-42.github.io/publications/2026/occufly/>.
- [8] Zhu Yu, Runmin Zhang, Jiacheng Ying, Junchen Yu, Xiaohai Hu, Lun Luo, Si-Yuan Cao, and Hui-Liang Shen. *Context and Geometry Aware Voxel Transformer for Semantic Scene Completion*. arXiv:2405.13675, 2024. <https://arxiv.org/abs/2405.13675>.
- [9] Shiyuan Chen, Wei Sui, Bohao Zhang, Zeyd Boukhers, John See, and Cong Yang. *Unleashing Semantic and Geometric Priors for 3D Scene Completion*. arXiv:2508.13601, 2025. <https://arxiv.org/abs/2508.13601>.
- [10] D-Robotics-AI-Lab. *FoundationSSC reference implementation*. GitHub repository. <https://github.com/D-Robotics-AI-Lab/FoundationSSC>.
- [11] Yiming Li, Zhiding Yu, Christopher Choy, Chaowei Xiao, Jose M. Alvarez, Sanja Fidler, Chen Feng, and Anima Anandkumar. *VoxFormer: Sparse Voxel Transformer for Camera-Based 3D Semantic Scene Completion*. CVPR 2023; arXiv:2302.12251. <https://arxiv.org/abs/2302.12251>.